Programa studentų pažymių tvarkymui

Sugeneruota Doxygen 1.13.2

skyrius 1

README

v1.5

Programa buvo papildyta nauja klase Human, o klasė StudentClass tapo išvestine iš jos. Abstrakčios klasės objektai negali būti sukurti:

v1.2

Šioje versijoje buvo realizuoti "Rule of Five" metodai:

1. Destruktorius: iškviečiamas, kai objektas nustoja egzistuoti, ir atlaisvina objekto užimamą atmintį. Štai jo realizacija:

```
StudentClass::~StudentClass() {
grades.clear();
name.clear();
surname.clear();

       }
```

2. Kopijavimo konstruktorius: yra skirtas sukurti naują objektą, kuris yra tiksli kopija jau egzistuojančio objekto:

```
StudentClass(const StudentClass &student) {
         this->name = student.name;
         this->surname = student.surname;
         this->grades = student.grades;
         this->exam_grade = student.exam_grade;
         this->final_grade = student.final_grade;
        }
```

3. Kopijavimo priskyrimo operatorius: naudojamas, kai jau egzistuojančiam objektui priskiriamos kito egzistuojančio objekto reikšmės:

```
StudentClass& operator=(const StudentClass &student) {
            if (this == &student) {
              return *this;
            }
            this->name = student.name;
            this->surname = student.surname;
            this->grades = student.grades;
            this->exam_grade = student.exam_grade;
            this->final_grade = student.final_grade;

            return *this;

          }
```

1. Perkėlimo konstruktorius: skirtas sukurti naują objektą, perkeliant visus resursus, t.y. senas objektas perduoda visas savo reikšmes naujai sukurtam objektui:

```
StudentClass(StudentClass&& student) noexcept
: name(move(student.name)),
surname(move(student.surname)),
grades(move(student.grades)),
exam_grade(student.exam_grade),
final_grade(student.final_grade)       {}
```

1. Perkėlimo priskyrimo operatorius: naudojamas, kai egzistuojančiam objektui priskiriama laikino (rvalue) objekto reikšmė, perkeliant resursus

```
StudentClass& operator=(StudentClass&& student) noexcept {
           if (this == &student) {
             return *this;
           }

           name = move(student.name);
           surname =move(student.surname);
           grades = move(student.grades);
           exam_grade = student.exam_grade;
           final_grade = student.final_grade;
           student.exam_grade = 0.0;
           student.final_grade = 0.0;

           return *this;
         }
```

"Rule of five" naudojimas pagerina kodo semantiką ir garantuoja efektyvų perkėlimą, kuris leidžia optimizuoti darbą su laikinais objektais. Taip pat šioje versijoje buvo padarytas įvesties/išvesties operatorių perdengimas. Šie nauji metodai leidžia suteikti naują ar papildomą funkcionalumą jau egzistuojantiems operatoriams (šiuo atveju tai yra $<<$ ir $>>$), kad jie sklandžiai veiktų su klase StudentClass. Apžvelkime išvesties operatoriaus perdengimą:

```
friend ostream& operator«(ostream& os, const StudentClass& student) {
            os « left
               « setw(15) « student.surname
               « setw(15) « student.name
               « right
               « setw(10) « fixed « setprecision(2) « student.final_grade;
            return os;
          }
```

Šis metodas skirtas supaprastinti StudentClass objektų infromacijos išvedimą tiek į konsolę, tiek į failą. Taigi dabar išvedimo sintaksė atrodo taip:

```
cout« student; //student - klasės StudentClass objektas
```

Taip pat yra panaudotas įvesties operatoriaus perdengimas:

```
friend istream& operator»(istream& is, StudentClass& student) {
            student = StudentClass();

            string line;
            if (!getline(is » ws, line)) return is;

            istringstream iss(line);

            string namePart, surnamePart;
            if (!(iss » namePart » surnamePart)) return is;

            student.name = namePart;
            student.surname = surnamePart;

            // Read grades
            vector<double> grades;
            double grade;
            while (iss » grade) {
              grades.push_back(grade);
            }

            if (!grades.empty()) {
              student.setExamGrades(grades.back());
              grades.pop_back();
              student.setGrades(grades);
              student.calculateFinalGradesAverageClass(student);
            }

            return is;
          }
```

Šis metodas yra universalus, t.y. tinka tiek įvedimui į konsolę, tiek įvedimui į failą. Jis gražina nuorodą į įvesties srautą is, kad būtų galima sujungti $>>$ operacijas.

Taigi, operatorių perdengimas yra naudingas, nes jis supaprastina kodą, nes visa įvesties/išvesties logika yra inkapsuliuota klasės, o ne išskaidyta po visą programą. Taip pat, jei šį kodą naudotų kiti programuotojai, tai supaprastintų jų darbą.

v1.1 Struct duomenų tipas pakeistas į klases. Atliktas spartos tyrimas atlikant įrašų nuskaitymą iš failo, rūšiavimą ir dalijimą naudojant trečią strategiją (iš praeito tyrimo) naudojant vector konteinerį ir lyginanama pagal skirtingas kompiliavimo flag'ais (pateikti 5 bandymų vidurkiai)

-O3 flag'as (.exe failo dydis 334KB)

Skaitymas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.2694	0.52
1000000	2.6156	1.109

Rūšiavimas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.236	0.142
1000000	2.4194	1.4534

Dalijimas į konteinerius

Įrašų skaičius faile	Klasės	Struktūros
100000	0.00891175	8.61754E-5
1000000	0.0840014832	0.0010766084

-O2 flag'as (.exe failo dydis 335KB)

Skaitymas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.2518	0.168
1000000	2.4036	1.7048

Rūšiavimas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.2168	0.1078
1000000	2.2218	1.1678

Dalijimas į konteinerius

Įrašų skaičius faile	Klasės	Struktūros
100000	0.0078411084	0.0046912498
1000000	0.074090567	0.0627567658

-O1 flag'as (failo dydis 812KB)

Skaitymas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.338	0.2178
1000000	3.0732	2.2194

Rūšiavimas:

Įrašų skaičius faile	Klasės	Struktūros
100000	0.287	0.145
1000000	2.9262	1.5338

Dalijimas į konteinerius

Įrašų skaičius faile	Klasės	Struktūros
100000	0.0102535416	0.0088353666
1000000	0.0951730582	0.1314340166

<<<<<<< HEAD

1.1 # OOP_2uzd

v1.0 Šio tyrimo tikslas yra ištirti programos spartą naudojant skirtingus konteinerius: vector, list, deque. Atlikti skirtingi veiksmai su konteineriais ir buvo matuojamas atlikimo veikimo laikas, pavaizduotas lentelėse:

1. Nuskaitymas iš failo į atitinkamą konteinerį.
   1.1. Rezultatai skaitymo į List konteinerį:

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.001	0.002	0.002	0.002	0.002	0.0018
10000	0.018	0.019	0.022	0.018	0.019	0.0192
100000	0.195	0.22	0.2	0.216	0.211	0.2084
1000000	2.023	2.013	2.009	2.012	2.018	2.015
10000000	20.616	20.63	20.564	20.446	20.402	20.5316

1.2. Rezultatai skaitymo į Deque konteinerį:

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.002	0.002	0.002	0.002	0.002	0.002
10000	0.02	0.021	0.021	0.021	0.021	0.0208
100000	0.214	0.216	0.223	0.219	0.224	0.2184
1000000	2.239	2.173	2.17	2.168	2.173	2.1846
10000000	27.198	25.904	25.995	26.309	26.451	26.3714

1.3 Rezultatai skaitymo į Vector konteinerį:

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.014	0.014	0.014	0.013	0.014	0.0138
10000	0.057	0.057	0.058	0.063	0.064	0.0598
100000	0.379	0.377	0.378	0.627	0.839	0.52
1000000	3.818	3.7	3.797	7.434	6.293	5.0084
10000000	40.703	40.693	40.651	68.402	66.579	51.4056

1. Studentų rūšiavimas didėjimo tvarka (pagal galutinį pažymį iš vidurkio)

   2.1 Rūšiavimas su List:

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	6.8375e-05	5.4833e-05	5.5208e-05	5.5625e-05	5.425e-05	5.76582E-5
10000	0.000776459	0.000714625	0.000778375	0.000696541	0.000743917	0.↩ 0007419834
100000	0.0101778	0.0134357	0.0186094	0.0109893	0.0101133	0.0126651
1000000	0.413999	0.396489	0.437618	0.505107	0.436495	0.4379416
10000000	8.34264	8.01193	8.17679	8.01488	8.13933	8.137114

2.2. Rūšiavimas su Deque:

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	1.792e-06	1.917e-06	1.875e-06	1.916e-06s	1.916e-06	1.8832E-6
10000	6.5625e-05	7.3292e-05	3e-05	6.0708e-05	6.8958e-05	5.97166E-5
100000	0.00140246	0.00144383	0.00110646	0.00239883	0.00141058	0.001552432
1000000	0.105023	0.0354815	0.0365577	0.0287554	0.0330828	0.04778008
10000000	1.07304	0.380618	0.399486	0.402701	0.349551	0.5210792

2.3. Rūšiavimas su Vector

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.001	0.001	0.001	0.001	0.001	0.018
10000	0.012	0.014	0.014	0.014	0.093	0.0968
100000	0.135	0.144	0.142	0.148	0.141	0.142
1000000	1.511	1.436	1.393	1.475	1.452	1.4534
10000000	18.33	17.973	17.974	17.814	17.834	17.985

1. Įrašų dalijimas į du konteinerius

   3.1. Įrašų dalijimas į List

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.000135208	0.001	0.001	0.001	0.001	0.018
10000	0.00145558	0.014	0.014	0.014	0.093	0.0968
100000	0.035963	0.144	0.142	0.148	0.141	0.52
1000000	0.508848	1.436	1.393	1.475	1.452	5.0084
10000000	4.90112	5.40402	5.09065	4.66368	5.0537	51.4056

3.2. Įrašų dalijimas į Deque

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.000129417	3.7417e-05	3.425e-05	2.8083e-05	2.6167e-05	5.10668e-05
10000	0.000269834	0.000264333	0.000286709	0.000259	0.00028225	0.↩ 0002724252
100000	0.00298096	0.00301658	0.003529	0.00297167	0.00314108	0.003127858
1000000	0.0361712	0.0391695	0.0389418	0.0382549	0.0374115	0.03798978
10000000	2.93958	3.10767	3.02076	3.08732	2.97793	3.026652

3.3. Įrašų dalijimas į Vector

Įrašų skaičius faile	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
1000	0.001	0.001	0.001	0.001	0.001	0.001
10000	0.007	0.007	0.007	0.011	0.011	0.0086
100000	0.062	0.063	0.063	0.103	0.101	0.0784
1000000	0.694	0.699	0.695	1.135	1.12	0.8686
10000000	6.554	6.594	6.566	10.4	10.341	8.091

Algoritmas, naudojamas praeitame tyrime, gali būti optimizuotas. Galime tuo įsitikinti naudodami 3 strategijas:

1 strategija - dalijimas į du konteinerius, kur tas pats studentas yra tiek bendrame studentų konteineryje, tiek "vargšiukų" arba "kietekų". Ši strategija jau buvo panaudota praeitame tyrime ir rezultatai yra užfiksuoti lentelėse viršuje. Tačiau ši strategija yra labai neefektyvi atminties panaudojimo atvėju. Užfiksuotas 10000000 įrašų skirstymas su deque:

2 strategija - bendro studentų konteinerio skaidymas panaudojant tik vieną konteinerį "vargšiukai". Tokiu būdu visi vagršiukai bus atskirame konteineryje, o likę studentai bendrame konteineryje bus "kietekai". Lentelėje pateikti tyrimo vidurkiai:

List:

Įrašų skaičius	Vidurkis
1000	0.00014743775
10000	0.00166201
100000	0.02433255
1000000	0.34394575
10000000	4.329365

Vector:

Įrašų skaičius	Vidurkis
1000	3.6875E-5
10000	0.0003421418
100000	0.0037645168
1000000	0.0591652584
10000000	0.7618154

Deque:

Įrašų skaičius	Vidurkis
1000	1.75832E-5
10000	0.0001462588
100000	0.002258808
1000000	0.34394575
10000000	0.543685

3 strategija - bus optimizuota antra strategija efektyvesnius algoritmus.

List:

Įrašų skaičius	Vidurkis
1000	1.775E-6
10000	5.67E-5
100000	0.001672314
1000000	0.07585252
10000000	0.8578546

Vector:

Įrašų skaičius	Vidurkis
1000	5.334E-7
10000	8.0086E-6
100000	8.61754E-5
1000000	0.0010766084
10000000	0.0113756404

Deque:

Įrašų skaičius	Vidurkis
1000	1.1498E-6
10000	1.41084E-5
100000	0.000241125
1000000	0.003637294
10000000	0.9234382

v0.4 Atlikti du tyrimai programos veikimo greičio analizei. Pirmasis tyrimas skirtas darbo su failais (failų sukurimas ir jo uždarymas) spartos analizei. Matuojamas skirtingo dydžio failų sukūrimo laikas. Tyrimo patikimumui bandymas buvo atliekamas penkis kartus. Bandymų rezultatai yra pateikti lentelėje (laikas skaičiuojamas sekundėmis):

Failo pavadinimas	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
students1000.txt	0.014	0.013	0.014	0.014	0.014	0.014
students10000.txt	0.071	0.071	0.071	0.07	0.071	0.71
students100000.txt	0.627	0.622	0.63	0.619	0.616	0.6318
students1000000.txt	6.186	6.207	6.347	6.2	6.238	6.2356
students10000000.txt	68.331	69.294	69.67	68.262	68.326	68.7766

Antrasis tyrimas skirtas duomenų apdorojimo spartos analizei. Atliekami šie veiksmai:

1. Duomenų nuskaitymas iš failo
2. Studentų rūšiavimas į dvi kategorijas
3. Surūšiuotų studentų išvedimą į naujus failus.

Be to, buvo matuojamas visos programos veikimo laikas.Kiekvienas žingsnis taip pat buvo kartojamas po 5 kartus ir pateiktas šių bandymų vidurkis. Rezultatai pateikiami lentelėje (laikas pateikiamas sekundėmis):

1000 įrašų:

	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
Duomenų nuskaitymas iš failo	0.014	0.014	0.014	0.013	0.014	0.0138
Duomenų rūšiavimas didėjimo tvarka	0.02	0.02	0.02	0.015	0.015	0.018
Studentų rūšiavimas į dvi kategorijas	0.001	0.001	0.001	0.001	0.001	0.001
Surūšiuotų studentų išvedimas į kietekų failą	0.003	0.003	0.003	0.003	0.003	0.003
Surūšiuotų studentų išvedimas į vagršiukų failą	0.002	0.002	0.002	0.002	0.002	0.002
Visos programos veikimo laikas	0.04	0.04	0.04	0.036	0.037	0.0386

10000 įrašų:

	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
Duomenų nuskaitymas iš failo	0.057	0.057	0.058	0.063	0.064	0.0598
Duomenų rūšiavimas didėjimo tvarka	0.099	0.099	0.098	0.095	0.093	0.0968
Studentų rūšiavimas į dvi kategorijas	0.007	0.007	0.007	0.011	0.011	0.0086
Surūšiuotų studentų išvedimas į kietekų failą	0.015	0.013	0.012	0.021	0.02	0.0162
Surūšiuotų studentų išvedimas į vagršiukų failą	0.011	0.009	0.009	0.015	0.014	0.0116
Visos programos veikimo laikas	0.189	0.185	0.184	0.036	0.037	0.1262

100000 įrašų:

	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
Duomenų nuskaitymas iš failo	0.379	0.377	0.378	0.627	0.839	0.52
Duomenų rūšiavimas didėjimo tvarka	0.626	0.59	0.587	0.957	0.953	0.7426
Studentų rūšiavimas į dvi kategorijas	0.062	0.063	0.063	0.103	0.101	0.0784
Surūšiuotų studentų išvedimas į kietekų failą	0.126	0.126	0.124	0.218	0.218	0.1624
Surūšiuotų studentų išvedimas į vagršiukų failą	0.09	0.089	0.089	0.151	0.145	0.1128
Visos programos veikimo laikas	2.002	1.861	1.262	2.088	2.29	1.9006

1000000 įrašų:

	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
Duomenų nuskaitymas iš failo	3.818	3.7	3.797	7.434	6.293	5.0084
Duomenų rūšiavimas didėjimo tvarka	5.846	5.778	5.755	10.024	9.641	7.4088
Studentų rūšiavimas į dvi kategorijas	0.694	0.699	0.695	1.135	1.12	0.8686
Surūšiuotų studentų išvedimas į kietekų failą	1.288	1.314	1.355	2.15	2.061	1.6336
Surūšiuotų studentų išvedimas į vagršiukų failą	0.901	0.973	0.891	1.477	1.458	1.14
Visos programos veikimo laikas	13.79	13.515	12.675	22.511	20.863	16.6708

10000000 įrašų:

	Pirmas bandymas	Antras bandymas	Trečias bandymas	Ketvirtas bandymas	Penktas bandymas	Vidurkis
Duomenų nuskaitymas iš failo	40.703	40.693	40.651	68.402	66.579	51.4056
Duomenų rūšiavimas didėjimo tvarka	60.468	60.957	60.41	97.318	96.196	75.0698
Studentų rūšiavimas į dvi kategorijas	6.554	6.594	6.566	10.4	10.341	8.091
Surūšiuotų studentų išvedimas į kietekų failą	14.762	15.332	15.472	22.336	21.592	17.8988
Surūšiuotų studentų išvedimas į vagršiukų failą	9.717	9.702	9.959	16.731	16.377	12.4972
Visos programos veikimo laikas	135.063	135.857	134.898	218.088	213.962	167.5736

409c246 (initial commit2)

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

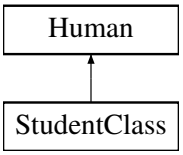
skyrius 5

Klasės Dokumentacija

5.1 Human Klasė

`#include <Human.h>`

Paveldimumo diagrama Human:



Vieši Metodai

- Human ()=default
- Human (string name, string surname)
- virtual ~Human ()=default
- virtual void printInfo ()=0
- Human (Human &&student) noexcept
- Human & operator= (Human &&other) noexcept
- string getName () const
- string getSurname () const
- const void setName (const string name)
- const void setSurname (const string surname)

Apsaugoti Atributai

- string name
- string surname

5.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.1.1 Human() [1/3]

`Human::Human () [default]`

5.1.1.2 Human() [2/3]

```
Human::Human (
            string name,
            string surname)
```

5.1.1.3 ~Human()

```
virtual Human::~Human ()  [virtual], [default]
```

5.1.1.4 Human() [3/3]

```
Human::Human (
            Human && student)  [inline], [noexcept]
```

5.1.2 Metodų Dokumentacija

5.1.2.1 getName()

```
string Human::getName () const  [inline]
```

5.1.2.2 getSurname()

```
string Human::getSurname () const  [inline]
```

5.1.2.3 operator=()

```
Human & Human::operator= (
            Human && other)  [inline], [noexcept]
```

5.1.2.4 printInfo()

```
virtual void Human::printInfo ()  [pure virtual]
```

Realizuota StudentClass.

5.1.2.5 setName()

```
const void Human::setName (
            const string name)  [inline]
```

5.1.2.6 setSurname()

```
const void Human::setSurname (
            const string surname)  [inline]
```

5.1.3 Atributų Dokumentacija

5.1.3.1 name

```
string Human::name  [protected]
```

5.1.3.2 surname

```
string Human::surname  [protected]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- Human.h
- Human.cpp

5.2 Student Struktūra

```
#include <Student.h>
```

Vieši Atributai

- std::string name
- std::string surname
- std::vector< uint8_t > grades
- uint8_t exam_grade
- double final_grade

5.2.1 Atributų Dokumentacija

5.2.1.1 exam_grade

```
uint8_t Student::exam_grade
```

5.2.1.2 final_grade

```
double Student::final_grade  [mutable]
```

5.2.1.3 grades

```
std::vector<uint8_t> Student::grades
```

5.2.1.4 name

```
std::string Student::name
```

5.2.1.5 surname

```
std::string Student::surname
```

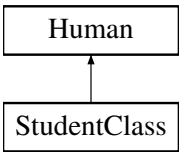
Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- Student.h

5.3 StudentClass Klasė

```
#include <StudentClass.h>
```

Paveldimumo diagrama StudentClass:



Vieši Metodai

- const vector< double > getGrades () const
- const double getExamGrades () const
- const void setExamGrades (const double grade)
- const void setGrades (const vector< double > grades)
- void clearGrades ()
- const void setFinalGrade (const double finalGrade)
- const double getFinalGrade () const
- vector< StudentClass > addStudentsObjects (vector< StudentClass > students)
- double averageClass (StudentClass &student)
- double medianClass (StudentClass &student)
- double calculateFinalGradesMedianClass (StudentClass &student)
- void calculateFinalGradesAverageClass (StudentClass &student)
- void printStudentListClass (vector< StudentClass > &students)
- void generateGradesClass (vector< StudentClass > &students)
- vector< string > loadFromFileClass (const string &filename)
- void writeStudentsToFile (const vector< StudentClass > &students, const string &filename)
- vector< StudentClass > readStudentsFileClass (const string &filename)
- vector< StudentClass > generateRandomStudentsClass (int count)

- vector< StudentClass > testClass ()
- bool compareByNameClass (StudentClass a, StudentClass b)
- bool compareBySurnameClass (StudentClass a, StudentClass b)
- bool compareByAverageClass (StudentClass a, StudentClass b)
- vector< StudentClass > sortByNameClass (vector< StudentClass > students)
- vector< StudentClass > sortBySurnameClass (vector< StudentClass > students)
- vector< StudentClass > sortByAverageClass (vector< StudentClass > students)
- void logDuration (const string &message, const std::chrono::high_resolution_clock::time_point &start, const std::chrono::high_resolution_clock::time_point &stop)
- void generateStudentsFileClass (int numberOfStudents)
- void sortStudentsInFileClass (vector< StudentClass > &students, int numberOfStudents)
- void strategyTwoVectorClass (vector< StudentClass > &students, vector< StudentClass > &vargsiukai, int num)
- void strategyThreeVector (vector< StudentClass > &students, vector< StudentClass > &vargsiukai, int num)
- StudentClass ()
- void printInfo () override
- StudentClass (string name, string surname, vector< double > grades, double exam_grade, double final↩ Grade)
- StudentClass (const StudentClass &student)
- StudentClass & operator= (const StudentClass &student)
- StudentClass (StudentClass &&student) noexcept
- StudentClass & operator= (StudentClass &&student) noexcept
- ~StudentClass ()

Vieši Metodai inherited from Human

- Human ()=default
- Human (string name, string surname)
- virtual ~Human ()=default
- Human (Human &&student) noexcept
- Human & operator= (Human &&other) noexcept
- string getName () const
- string getSurname () const
- const void setName (const string name)
- const void setSurname (const string surname)

Privatūs Atributai

- string name
- string surname
- ::vector< double > grades
- double exam_grade
- double final_grade

Draugai

- ostream & operator<< (ostream &os, const StudentClass &student)
- istream & operator>> (istream &is, StudentClass &student)

Additional Inherited Members

Apsaugoti Atributai inherited from Human

- string name
- string surname

5.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.3.1.1 StudentClass() [1/4]

```
StudentClass::StudentClass ()  [inline]
```

5.3.1.2 StudentClass() [2/4]

```
StudentClass::StudentClass (
            string name,
            string surname,
            vector< double > grades,
            double exam_grade,
            double finalGrade)  [inline]
```

5.3.1.3 StudentClass() [3/4]

```
StudentClass::StudentClass (
            const StudentClass & student)  [inline]
```

5.3.1.4 StudentClass() [4/4]

```
StudentClass::StudentClass (
            StudentClass && student)  [inline], [noexcept]
```

5.3.1.5 ~StudentClass()

```
StudentClass::~StudentClass ()  [inline]
```

5.3.2 Metodų Dokumentacija

5.3.2.1 addStudentsObjects()

```
vector< StudentClass > StudentClass::addStudentsObjects (
            vector< StudentClass > students)
```

5.3.2.2 averageClass()

```
double StudentClass::averageClass (
            StudentClass & student)
```

5.3.2.3 calculateFinalGradesAverageClass()

```
void StudentClass::calculateFinalGradesAverageClass (
            StudentClass & student)
```

5.3.2.4 calculateFinalGradesMedianClass()

```
double StudentClass::calculateFinalGradesMedianClass (
            StudentClass & student)
```

5.3.2.5 clearGrades()

```
void StudentClass::clearGrades ()  [inline]
```

5.3.2.6 compareByAverageClass()

```
bool StudentClass::compareByAverageClass (
            StudentClass a,
            StudentClass b)
```

5.3.2.7 compareByNameClass()

```
bool StudentClass::compareByNameClass (
            StudentClass a,
            StudentClass b)
```

5.3.2.8 compareBySurnameClass()

```
bool StudentClass::compareBySurnameClass (
            StudentClass a,
            StudentClass b)
```

5.3.2.9 generateGradesClass()

```
void StudentClass::generateGradesClass (
            vector< StudentClass > & students)
```

5.3.2.10 generateRandomStudentsClass()

```
vector< StudentClass > StudentClass::generateRandomStudentsClass (
            int count)
```

5.3.2.11 generateStudentsFileClass()

```
void StudentClass::generateStudentsFileClass (
            int numberOfStudents)
```

5.3.2.12 getExamGrades()

```
const double StudentClass::getExamGrades () const  [inline]
```

5.3.2.13 getFinalGrade()

```
const double StudentClass::getFinalGrade () const  [inline]
```

5.3.2.14 getGrades()

```
const vector< double > StudentClass::getGrades () const  [inline]
```

5.3.2.15 loadFromFileClass()

```
vector< string > StudentClass::loadFromFileClass (
            const string & filename)
```

5.3.2.16 logDuration()

```
void StudentClass::logDuration (
            const string & message,
            const std::chrono::high_resolution_clock::time_point & start,
            const std::chrono::high_resolution_clock::time_point & stop)
```

5.3.2.17 medianClass()

```
double StudentClass::medianClass (
            StudentClass & student)
```

5.3.2.18 operator=() [1/2]

```
StudentClass & StudentClass::operator= (
            const StudentClass & student)  [inline]
```

5.3.2.19 operator=() [2/2]

```
StudentClass & StudentClass::operator= (
            StudentClass && student)  [inline], [noexcept]
```

5.3.2.20 printInfo()

```
void StudentClass::printInfo ()  [inline], [override], [virtual]
```

Realizuoja Human.

5.3.2.21 printStudentListClass()

```
void StudentClass::printStudentListClass (
            vector< StudentClass > & students)
```

5.3.2.22 readStudentsFileClass()

```
vector< StudentClass > StudentClass::readStudentsFileClass (
            const string & filename)
```

5.3.2.23 setExamGrades()

```
const void StudentClass::setExamGrades (
            const double grade)  [inline]
```

5.3.2.24 setFinalGrade()

```
const void StudentClass::setFinalGrade (
            const double finalGrade)  [inline]
```

5.3.2.25 setGrades()

```
const void StudentClass::setGrades (
            const vector< double > grades)  [inline]
```

5.3.2.26 sortByAverageClass()

```
vector< StudentClass > StudentClass::sortByAverageClass (
            vector< StudentClass > students)
```

5.3.2.27 sortByNameClass()

```
vector< StudentClass > StudentClass::sortByNameClass (
            vector< StudentClass > students)
```

5.3.2.28 sortBySurnameClass()

```
vector< StudentClass > StudentClass::sortBySurnameClass (
            vector< StudentClass > students)
```

5.3.2.29 sortStudentsInFileClass()

```
void StudentClass::sortStudentsInFileClass (
            vector< StudentClass > & students,
            int numberOfStudents)
```

5.3.2.30 strategyThreeVector()

```
void StudentClass::strategyThreeVector (
            vector< StudentClass > & students,
            vector< StudentClass > & vargsiukai,
            int num)
```

5.3.2.31 strategyTwoVectorClass()

```
void StudentClass::strategyTwoVectorClass (
            vector< StudentClass > & students,
            vector< StudentClass > & vargsiukai,
            int num)
```

5.3.2.32 testClass()

```
vector< StudentClass > StudentClass::testClass ()
```

5.3.2.33 writeStudentsToFile()

```
void StudentClass::writeStudentsToFile (
            const vector< StudentClass > & students,
            const string & filename)
```

5.3.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.3.3.1 operator<<

```
ostream & operator<< (
            ostream & os,
            const StudentClass & student)  [friend]
```

5.3.3.2 operator>>

```
istream & operator>> (
            istream & is,
            StudentClass & student)  [friend]
```

5.3.4 Atributų Dokumentacija

5.3.4.1 exam_grade

```
double StudentClass::exam_grade  [private]
```

5.3.4.2 final_grade

```
double StudentClass::final_grade  [mutable], [private]
```

5.3.4.3 grades

```
::vector<double> StudentClass::grades  [private]
```

5.3.4.4 name

```
string StudentClass::name  [private]
```

5.3.4.5 surname

```
string StudentClass::surname  [private]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- StudentClass.h
- StudentClass.cpp

5.4 StudentDeque Struktūra

```
#include <StudentDeque.h>
```

Vieši Atributai

- std::string name
- std::string surname
- std::deque< double > grades
- int exam_grade
- double final_grade

5.4.1 Atributų Dokumentacija

5.4.1.1 exam_grade

```
int StudentDeque::exam_grade
```

5.4.1.2 final_grade

```
double StudentDeque::final_grade  [mutable]
```

5.4.1.3 grades

```
std::deque<double> StudentDeque::grades
```

5.4.1.4 name

```
std::string StudentDeque::name
```

5.4.1.5 surname

```
std::string StudentDeque::surname
```

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- StudentDeque.h

5.5 StudentList Struktūra

```
#include <StudentList.h>
```

Vieši Atributai

- std::string name
- std::string surname
- std::list< double > grades
- int exam_grade
- double final_grade

5.5.1 Atributų Dokumentacija

5.5.1.1 exam_grade

```
int StudentList::exam_grade
```

5.5.1.2 final_grade

```
double StudentList::final_grade  [mutable]
```

5.5.1.3 grades

```
std::list<double> StudentList::grades
```

5.5.1.4 name

```
std::string StudentList::name
```

5.5.1.5 surname

```
std::string StudentList::surname
```

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- StudentList.h

skyrius 6

Failo Dokumentacija

6.1 cmake-build-debug/CMakeFiles/3.30.5/CompilerIdC/CMake↩ CCompilerId.c Failo Nuoroda

Apibrėžimai

- #define __has_include(x)
- #define COMPILER_ID ""
- #define STRINGIFY_HELPER(X)
- #define STRINGIFY(X)
- #define PLATFORM_ID
- #define ARCHITECTURE_ID
- #define DEC(n)
- #define HEX(n)
- #define C_STD_99 199901L
- #define C_STD_11 201112L
- #define C_STD_17 201710L
- #define C_STD_23 202311L
- #define C_VERSION

Funkcijos

- int main (int argc, char ∗argv[ ])

Kintamieji

- char const ∗ info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
- char const ∗ info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
- char const ∗ info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
- const char ∗ info_language_standard_default
- const char ∗ info_language_extensions_default

6.1.1 Apibrėžimų Dokumentacija

6.1.1.1 __has_include

```
#define __has_include(
              x)
```

Reikšmė:

```
0
```

6.1.1.2 ARCHITECTURE_ID

```
#define ARCHITECTURE_ID
```

6.1.1.3 C_STD_11

```
#define C_STD_11 201112L
```

6.1.1.4 C_STD_17

```
#define C_STD_17 201710L
```

6.1.1.5 C_STD_23

```
#define C_STD_23 202311L
```

6.1.1.6 C_STD_99

```
#define C_STD_99 199901L
```

6.1.1.7 C_VERSION

```
#define C_VERSION
```

6.1.1.8 COMPILER_ID

```
#define COMPILER_ID ""
```

6.1.1.9 DEC

```
#define DEC(
               n)
```

Reikšmė:

```
  ('0' + (((n) / 10000000)%10)), \
  ('0' + (((n) / 1000000)%10)),  \
  ('0' + (((n) / 100000)%10)),   \
  ('0' + (((n) / 10000)%10)),    \
  ('0' + (((n) / 1000)%10)),     \
  ('0' + (((n) / 100)%10)),      \
  ('0' + (((n) / 10)%10)),       \
  ('0' +  ((n) % 10))
```

6.1.1.10 HEX

```
#define HEX(
               n)
```

Reikšmė:

```
  ('0' + ((n)»28 & 0xF)), \
  ('0' + ((n)»24 & 0xF)), \
  ('0' + ((n)»20 & 0xF)), \
  ('0' + ((n)»16 & 0xF)), \
  ('0' + ((n)»12 & 0xF)), \
  ('0' + ((n)»8  & 0xF)), \
  ('0' + ((n)»4  & 0xF)), \
  ('0' + ((n)     & 0xF))
```

6.1.1.11 PLATFORM_ID

```
#define PLATFORM_ID
```

6.1.1.12 STRINGIFY

```
#define STRINGIFY(
               X)
```

Reikšmė:

```
STRINGIFY_HELPER(X)
```

6.1.1.13 STRINGIFY_HELPER

```
#define STRINGIFY_HELPER(
               X)
```

Reikšmė:

```
#X
```

6.1.2 Funkcijos Dokumentacija

6.1.2.1 main()

```
int main (
             int argc,
             char * argv[])
```

6.1.3 Kintamojo Dokumentacija

6.1.3.1 info_arch

```
char const* info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
```

6.1.3.2 info_compiler

```
char const* info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
```

6.1.3.3 info_language_extensions_default

```
const char* info_language_extensions_default
```

Pradinė reikšmė:

```
= "INFO" ":" "extensions_default["



  "OFF"

"]"
```

6.1.3.4 info_language_standard_default

```
const char* info_language_standard_default
```

Pradinė reikšmė:

```
=
  "INFO" ":" "standard_default[" C_VERSION "]"
```

6.1.3.5 info_platform

```
char const* info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
```

6.2 cmake-build-debug/CMakeFiles/3.30.5/CompilerIdCXX/CMake↩ CXXCompilerId.cpp Failo Nuoroda

Apibrėžimai

- #define __has_include(x)
- #define COMPILER_ID ""
- #define STRINGIFY_HELPER(X)
- #define STRINGIFY(X)
- #define PLATFORM_ID
- #define ARCHITECTURE_ID
- #define DEC(n)
- #define HEX(n)
- #define CXX_STD_98 199711L
- #define CXX_STD_11 201103L
- #define CXX_STD_14 201402L
- #define CXX_STD_17 201703L
- #define CXX_STD_20 202002L
- #define CXX_STD_23 202302L
- #define CXX_STD __cplusplus

Funkcijos

- int main (int argc, char ∗argv[ ])

Kintamieji

- char const ∗ info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
- char const ∗ info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
- char const ∗ info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
- const char ∗ info_language_standard_default
- const char ∗ info_language_extensions_default

6.2.1 Apibrėžimų Dokumentacija

6.2.1.1 __has_include

```
#define __has_include(
              x)
```

Reikšmė:

```
0
```

6.2.1.2 ARCHITECTURE_ID

```
#define ARCHITECTURE_ID
```

6.2.1.3 COMPILER_ID

```
#define COMPILER_ID ""
```

6.2.1.4 CXX_STD

```
#define CXX_STD __cplusplus
```

6.2.1.5 CXX_STD_11

```
#define CXX_STD_11 201103L
```

6.2.1.6 CXX_STD_14

```
#define CXX_STD_14 201402L
```

6.2.1.7 CXX_STD_17

```
#define CXX_STD_17 201703L
```

6.2.1.8 CXX_STD_20

```
#define CXX_STD_20 202002L
```

6.2.1.9 CXX_STD_23

```
#define CXX_STD_23 202302L
```

6.2.1.10 CXX_STD_98

```
#define CXX_STD_98 199711L
```

6.2.1.11 DEC

```
#define DEC(
              n)
```

Reikšmė:

```
  ('0' + (((n) / 10000000)%10)), \
  ('0' + (((n) / 1000000)%10)),  \
  ('0' + (((n) / 100000)%10)),   \
  ('0' + (((n) / 10000)%10)),    \
  ('0' + (((n) / 1000)%10)),     \
  ('0' + (((n) / 100)%10)),      \
  ('0' + (((n) / 10)%10)),       \
  ('0' +  ((n) % 10))
```

6.2.1.12 HEX

```
#define HEX(
              n)
```

Reikšmė:

```
  ('0' + ((n)»28 & 0xF)), \
  ('0' + ((n)»24 & 0xF)), \
  ('0' + ((n)»20 & 0xF)), \
  ('0' + ((n)»16 & 0xF)), \
  ('0' + ((n)»12 & 0xF)), \
  ('0' + ((n)»8  & 0xF)), \
  ('0' + ((n)»4  & 0xF)), \
  ('0' + ((n)     & 0xF))
```

6.2.1.13 PLATFORM_ID

```
#define PLATFORM_ID
```

6.2.1.14 STRINGIFY

```
#define STRINGIFY(
              X)
```

Reikšmė:

```
STRINGIFY_HELPER(X)
```

6.2.1.15 STRINGIFY_HELPER

```
#define STRINGIFY_HELPER(
              X)
```

Reikšmė:

```
#X
```

6.2.2 Funkcijos Dokumentacija

6.2.2.1 main()

```
int main (
             int argc,
             char * argv[])
```

6.2.3 Kintamojo Dokumentacija

6.2.3.1 info_arch

```
char const* info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
```

6.2.3.2 info_compiler

```
char const* info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
```

6.2.3.3 info_language_extensions_default

```
const char* info_language_extensions_default
```

Pradinė reikšmė:

```
= "INFO" ":" "extensions_default["



  "OFF"
"]"
```

6.2.3.4 info_language_standard_default

```
const char* info_language_standard_default
```

Pradinė reikšmė:

```
= "INFO" ":" "standard_default["









  "98"
"]"
```

6.2.3.5 info_platform

```
char const* info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
```

6.3 functions.cpp Failo Nuoroda

```
#include "functions.h"
```

Funkcijos

- vector< Student > addStudents (vector< Student > students)
- double average (const Student &student)
- double median (const Student &student)
- double calculateFinalGradesMedian (const Student &student)
- void calculateFinalGradesAverage (const Student &student)
- void printStudentList (vector< Student > &students)
- void generateGrades (vector< Student > &students)
- vector< string > loadFromFile (const string &filename)
- vector< Student > readStudentsFile (const string &filename)
- vector< Student > generateRandomStudents (int count)
- vector< Student > test ()
- bool compareByName (const Student a, const Student b)
- bool compareBySurname (const Student a, const Student b)
- bool compareByAverage (const Student a, const Student b)
- bool compareByMedian (const Student a, const Student b)
- vector< Student > sortByName (vector< Student > students)
- vector< Student > sortBySurname (vector< Student > students)
- vector< Student > sortByAverage (vector< Student > students)
- vector< Student > sortByMedian (vector< Student > students)
- void logDuration (const string &message, const high_resolution_clock::time_point &start, const high_↩ resolution_clock::time_point &stop)
- void generateStudentsFile (int numberOfStudents)
- void sortStudentsInFile (vector< Student > &students, int numberOfStudents)
- void strategyTwoVector (vector< Student > &students, vector< Student > &vargsiukai, int num)
- void strategyThreeVector (vector< Student > &students, vector< Student > &vargsiukai, int num)

6.3.1 Funkcijos Dokumentacija

6.3.1.1 addStudents()

```
vector< Student > addStudents (
            vector< Student > students)
```

6.3.1.2 average()

```
double average (
            const Student & student)
```

6.3.1.3 calculateFinalGradesAverage()

```
void calculateFinalGradesAverage (
            const Student & student)
```

6.3.1.4 calculateFinalGradesMedian()

```
double calculateFinalGradesMedian (
            const Student & student)
```

6.3.1.5 compareByAverage()

```
bool compareByAverage (
            const Student a,
            const Student b)
```

6.3.1.6 compareByMedian()

```
bool compareByMedian (
            const Student a,
            const Student b)
```

6.3.1.7 compareByName()

```
bool compareByName (
            const Student a,
            const Student b)
```

6.3.1.8 compareBySurname()

```
bool compareBySurname (
            const Student a,
            const Student b)
```

6.3.1.9 generateGrades()

```
void generateGrades (
            vector< Student > & students)
```

6.3.1.10 generateRandomStudents()

```
vector< Student > generateRandomStudents (
            int count)
```

6.3.1.11 generateStudentsFile()

```
void generateStudentsFile (
            int numberOfStudents)
```

6.3.1.12 loadFromFile()

```
vector< string > loadFromFile (
            const string & filename)
```

6.3.1.13 logDuration()

```
void logDuration (
            const string & message,
            const high_resolution_clock::time_point & start,
            const high_resolution_clock::time_point & stop)
```

6.3.1.14 median()

```
double median (
            const Student & student)
```

6.3.1.15 printStudentList()

```
void printStudentList (
            vector< Student > & students)
```

6.3.1.16 readStudentsFile()

```
vector< Student > readStudentsFile (
            const string & filename)
```

6.3.1.17 sortByAverage()

```
vector< Student > sortByAverage (
            vector< Student > students)
```

6.3.1.18 sortByMedian()

```
vector< Student > sortByMedian (
            vector< Student > students)
```

6.3.1.19 sortByName()

```
vector< Student > sortByName (
            vector< Student > students)
```

6.3.1.20 sortBySurname()

```
vector< Student > sortBySurname (
            vector< Student > students)
```

6.3.1.21 sortStudentsInFile()

```
void sortStudentsInFile (
            vector< Student > & students,
            int numberOfStudents)
```

6.3.1.22 strategyThreeVector()

```
void strategyThreeVector (
            vector< Student > & students,
            vector< Student > & vargsiukai,
            int num)
```

6.3.1.23 strategyTwoVector()

```
void strategyTwoVector (
            vector< Student > & students,
            vector< Student > & vargsiukai,
            int num)
```

6.3.1.24 test()

```
vector< Student > test ()
```

6.4 functions.h Failo Nuoroda

```
#include "Student.h"
#include <fstream>
#include <sstream>
```

Funkcijos

- vector< Student > addStudents (vector< Student > students)
- double average (const Student &student)
- double median (const Student &student)
- double calculateFinalGradesMedian (const Student &student)
- void calculateFinalGradesAverage (const Student &student)
- void printStudentList (vector< Student > &students)
- void generateGrades (vector< Student > &students)
- vector< string > loadFromFile (const string &fileName)
- vector< Student > readStudentsFile (const string &fileName)
- vector< Student > generateRandomStudents (int count)
- vector< Student > test ()
- bool compareByName (const Student a, const Student b)
- bool compareBySurname (const Student a, const Student b)
- bool compareByAverage (const Student a, const Student b)
- bool compareByMedian (const Student a, const Student b)
- vector< Student > sortByName (vector< Student > students)
- vector< Student > sortBySurname (vector< Student > students)
- vector< Student > sortByAverage (vector< Student > students)
- vector< Student > sortByMedian (vector< Student > students)
- void logDuration (const string &message, const auto &start, const auto &stop)
- void generateStudentsFile (int numberOfStudents)
- void sortStudentsInFile (vector< Student > &students, int numberOfStudents)
- void strategyTwoVector (vector< Student > &students, vector< Student > &vargsiukai, int num)
- void strategyThreeVector (vector< Student > &students, vector< Student > &vargsiukai, int num)

6.4.1 Funkcijos Dokumentacija

6.4.1.1 addStudents()

```
vector< Student > addStudents (
            vector< Student > students)
```

6.4.1.2 average()

```
double average (
            const Student & student)
```

6.4.1.3 calculateFinalGradesAverage()

```
void calculateFinalGradesAverage (
            const Student & student)
```

6.4.1.4 calculateFinalGradesMedian()

```
double calculateFinalGradesMedian (
            const Student & student)
```

6.4.1.5 compareByAverage()

```
bool compareByAverage (
            const Student a,
            const Student b)
```

6.4.1.6 compareByMedian()

```
bool compareByMedian (
            const Student a,
            const Student b)
```

6.4.1.7 compareByName()

```
bool compareByName (
            const Student a,
            const Student b)
```

6.4.1.8 compareBySurname()

```
bool compareBySurname (
            const Student a,
            const Student b)
```

6.4.1.9 generateGrades()

```
void generateGrades (
            vector< Student > & students)
```

6.4.1.10 generateRandomStudents()

```
vector< Student > generateRandomStudents (
            int count)
```

6.4.1.11 generateStudentsFile()

```
void generateStudentsFile (
            int numberOfStudents)
```

6.4.1.12 loadFromFile()

```
vector< string > loadFromFile (
            const string & fileName)
```

6.4.1.13 logDuration()

```
void logDuration (
            const string & message,
            const auto & start,
            const auto & stop)
```

6.4.1.14 median()

```
double median (
            const Student & student)
```

6.4.1.15 printStudentList()

```
void printStudentList (
            vector< Student > & students)
```

6.4.1.16 readStudentsFile()

```
vector< Student > readStudentsFile (
            const string & fileName)
```

6.4.1.17 sortByAverage()

```
vector< Student > sortByAverage (
            vector< Student > students)
```

6.4.1.18 sortByMedian()

```
vector< Student > sortByMedian (
            vector< Student > students)
```

6.4.1.19 sortByName()

```
vector< Student > sortByName (
            vector< Student > students)
```

6.4.1.20 sortBySurname()

```
vector< Student > sortBySurname (
            vector< Student > students)
```

6.4.1.21 sortStudentsInFile()

```
void sortStudentsInFile (
            vector< Student > & students,
            int numberOfStudents)
```

6.4.1.22 strategyThreeVector()

```
void strategyThreeVector (
            vector< Student > & students,
            vector< Student > & vargsiukai,
            int num)
```

6.4.1.23 strategyTwoVector()

```
void strategyTwoVector (
            vector< Student > & students,
            vector< Student > & vargsiukai,
            int num)
```

6.4.1.24 test()

```
vector< Student > test ()
```

6.5 functions.h

Eiti į šio failo dokumentaciją.

```
00001 //
00002 // Created by Anastasija Fedorenko on 2025-02-23.
00003 //
00004
00005 #ifndef FUNCTIONS_H
00006 #define FUNCTIONS_H
00007
00008 #include "Student.h"
00009 #include <fstream>
00010 #include <sstream>
00011 using namespace std;
00012 using namespace std::chrono;
00013
00014 vector<Student> addStudents(vector<Student> students);
00015 double average(const Student &student);
00016 double median(const Student &student);
00017 double calculateFinalGradesMedian(const Student &student);
00018 void calculateFinalGradesAverage(const Student &student);
00019 void printStudentList(vector<Student> &students);
00020 void generateGrades(vector<Student> &students);
00021 vector<string> loadFromFile(const string &fileName);
00022 vector<Student> readStudentsFile(const string &fileName);
00023 vector<Student> generateRandomStudents(int count);
00024 vector<Student> test();
00025 bool compareByName(const Student a, const Student b);
00026 bool compareBySurname(const Student a, const Student b);
00027 bool compareByAverage(const Student a, const Student b);
00028 bool compareByMedian(const Student a, const Student b);
00029 vector<Student> sortByName(vector<Student> students);
00030 vector<Student> sortBySurname(vector<Student> students);
00031 vector<Student> sortByAverage(vector<Student> students);
00032 vector<Student> sortByMedian(vector<Student> students);
00033 void logDuration(const string& message, const auto& start, const auto& stop);
00034 void generateStudentsFile(int numberOfStudents);
00035 void sortStudentsInFile(vector<Student>& students,int numberOfStudents);
00036 void strategyTwoVector(vector<Student>& students, vector<Student>& vargsiukai, int num);
00037 void strategyThreeVector(vector<Student>& students, vector<Student>& vargsiukai, int num);
00038
00039
00040
00041 #endif //FUNCTIONS_H
```

6.6 Human.cpp Failo Nuoroda

```
#include "Human.h"
```

6.7 Human.h Failo Nuoroda

```
#include <string>
#include <iostream>
```

Klasės

- class Human

6.8 Human.h

Eiti į šio failo dokumentaciją.

```
00001 //
00002 // Created by Anastasija Fedorenko on 2025-04-21.
00003 //
00004
00005 #ifndef HUMAN_H
00006 #define HUMAN_H
00007
00008 #include <string>
00009 #include <iostream>
00010 using namespace std;
00011 class Human {
00012 protected:
00013     string name;
00014     string surname;
00015
00016 public:
00017     Human() = default;
00018     Human(string name, string surname);
00019     virtual ~Human() =default;
00020
00021     virtual void printInfo() = 0;
00022
00023     Human(Human&& student) noexcept : name(move(student.name)), surname(move(student.surname)) {}
00024
00025     Human& operator=(Human&& other) noexcept {
00026         if (this == &other) return *this;
00027         name = std::move(other.name);
00028         surname = std::move(other.surname);
00029         return *this;
00030     }
00031
00032
00033     string getName()const{return name;}
00034     string getSurname()const{return surname;}
00035     const void setName(const string name){this->name = name;}
00036     const void setSurname(const string surname){this->surname = surname;}
00037 };
00038
00039 #endif
```

6.9 README.md Failo Nuoroda

6.10 Student.cpp Failo Nuoroda

```
#include "functions.h"
#include "StudentList.h"
#include "StudentDeque.h"
#include "StudentClass.h"
#include "Human.h"
```

Funkcijos

- int main ()

6.10.1 Funkcijos Dokumentacija

6.10.1.1 main()

```
int main ()
```

6.11 Student.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <iostream>
#include <numeric>
#include <iomanip>
```

Klasės

- struct Student

6.12 Student.h

Eiti į šio failo dokumentaciją.

```
00001
00002
00003 #ifndef STUDENT_H
00004 #define STUDENT_H
00005
00006 #include <string>
00007 #include <vector>
00008 #include <iostream>
00009 #include <numeric>
00010 #include <iomanip>
00011
00012 struct Student {
00013     std::string name;
00014     std::string surname;
00015     std::vector<uint8_t> grades;
00016     uint8_t exam_grade;
00017     mutable double final_grade;
00018
00019 };
00020
00021
00022
00023 #endif //STUDENT_H
```

6.13 StudentClass.cpp Failo Nuoroda

```
#include "functions.h"
#include "StudentClass.h"
```

Funkcijos

- inline ::ostream & operator<< (::ostream &os, const ::vector< StudentClass > &students)

6.13.1 Funkcijos Dokumentacija

6.13.1.1 operator<<()

```
inline ::ostream & operator<< (
            ::ostream & os,
            const ::vector< StudentClass > & students)
```

6.14 StudentClass.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <iostream>
#include <numeric>
#include <iomanip>
#include <fstream>
#include <sstream>
#include <algorithm>
#include "Human.h"
```

Klasės

- class StudentClass

6.15 StudentClass.h

Eiti į šio failo dokumentaciją.

```
00001 //
00002 // Created by Anastasija Fedorenko on 2025-03-28.
00003 //
00004
00005 #ifndef STUDENTCLASS_H
00006 #define STUDENTCLASS_H
00007 #include <string>
00008 #include <vector>
00009 #include <iostream>
00010 #include <numeric>
00011 #include <iomanip>
00012 #include <fstream>
00013 #include <sstream>
00014 #include <algorithm>
```

```
00015 #include "Human.h"
00016 using namespace std;
00017
00018     class StudentClass : public Human {
00019         public:
00020
00021
00022
00023             const vector<double> getGrades()const{return grades;}
00024             const double getExamGrades()const{return exam_grade;}
00025             const void setExamGrades(const double grade){this->exam_grade = grade;}
00026             const void setGrades(const vector<double> grades){this->grades = grades;}
00027             void clearGrades(){this->grades.clear();}
00028             const void setFinalGrade(const double finalGrade){this->final_grade = finalGrade;}
00029             const double getFinalGrade()const{return final_grade;}
00030             vector<StudentClass> addStudentsObjects(vector<StudentClass> students);
00031             double averageClass(StudentClass &student);
00032             double medianClass(StudentClass &student);
00033           double calculateFinalGradesMedianClass(StudentClass &student);
00034             void calculateFinalGradesAverageClass(StudentClass &student);
00035           void printStudentListClass(vector<StudentClass> &students);
00036             void generateGradesClass(vector<StudentClass> &students);
00037         vector<string> loadFromFileClass(const string &filename);
00038       void writeStudentsToFile(const vector<StudentClass>& students, const string& filename);
00039           vector<StudentClass> readStudentsFileClass(const string &filename);
00040         vector<StudentClass> generateRandomStudentsClass(int count);
00041           vector<StudentClass> testClass();
00042         bool compareByNameClass(StudentClass a, StudentClass b);
00043         bool compareBySurnameClass(StudentClass a, StudentClass b);
00044         bool compareByAverageClass(StudentClass a, StudentClass b);
00045         vector<StudentClass> sortByNameClass(vector<StudentClass> students);
00046         vector<StudentClass> sortBySurnameClass(vector<StudentClass> students);
00047         vector<StudentClass> sortByAverageClass(vector<StudentClass> students);
00048         void logDuration(const string& message, const std::chrono::high_resolution_clock::time_point&
      start, const std::chrono::high_resolution_clock::time_point& stop);
00049         void generateStudentsFileClass(int numberOfStudents);
00050         void sortStudentsInFileClass(vector<StudentClass>& students,int numberOfStudents);
00051         void strategyTwoVectorClass(vector<StudentClass>& students, vector<StudentClass>& vargsiukai,
      int num);
00052         void strategyThreeVector(vector<StudentClass>& students, vector<StudentClass>& vargsiukai, int
      num);
00053
00054
00055
00056           //default konstruktorius
00057       StudentClass() : Human(), grades{}, exam_grade(0), final_grade(0) {}
00058
00059       void printInfo() override {
00060           cout « "Vardas: " « getName() « ", Pavarde: " « getSurname()
00061                « ", Gal. pazymys: " « fixed « setprecision(2) « getFinalGrade() « endl;
00062       }
00063
00064             //konstruktorius
00065       StudentClass(string name, string surname, vector<double> grades, double exam_grade, double
      finalGrade)
00066           : Human(name, surname),
00067             grades(grades),
00068             exam_grade(exam_grade),
00069             final_grade(finalGrade) {}
00070
00071
00072           //copy konstruktorius
00073       StudentClass(const StudentClass &student)
00074     : Human(student.getName(), student.getSurname()),
00075       grades(student.grades),
00076       exam_grade(student.exam_grade),
00077       final_grade(student.final_grade) {}
00078   //copy asignment operator
00079       StudentClass& operator=(const StudentClass &student) {
00080         if (this == &student) return *this;
00081         Human::setName(student.getName());
00082         Human::setSurname(student.getSurname());
00083         grades = student.grades;
00084         exam_grade = student.exam_grade;
00085         final_grade = student.final_grade;
00086         return *this;
00087       }
00088       //move konstruktorius
00089       StudentClass(StudentClass&& student) noexcept
00090         : Human(move(student)),
00091           grades(move(student.grades)),
00092           exam_grade(student.exam_grade),
00093           final_grade(student.final_grade) {
00094         student.exam_grade = 0.0;
00095         student.final_grade = 0.0;
00096         student.grades.clear();
00097         student.name.clear();
```

```
00098         student.surname.clear();
00099       }
00100
00101       //move assignment operatorius
00102       StudentClass& operator=(StudentClass&& student) noexcept {
00103         if (this == &student) return *this;
00104         Human::operator=(move(student));
00105         grades = move(student.grades);
00106         exam_grade = student.exam_grade;
00107         final_grade = student.final_grade;
00108         student.exam_grade = 0;
00109         student.final_grade = 0;
00110         return *this;
00111       }
00112
00113       friend ostream& operator«(ostream& os, const StudentClass& student) {
00114
00115             os « left
00116                « setw(15) « student.getSurname()
00117                « setw(15) « student.getName()
00118                « right
00119                « setw(10) « fixed « setprecision(2) « student.getFinalGrade();
00120             return os;
00121           }
00122
00123       friend istream& operator»(istream& is, StudentClass& student) {
00124             student = StudentClass();
00125
00126             string line;
00127             if (!getline(is » ws, line)) return is;
00128
00129             istringstream iss(line);
00130
00131             string namePart, surnamePart;
00132             if (!(iss » namePart » surnamePart)) return is;
00133
00134             student.setName(namePart);
00135             student.setSurname(surnamePart);
00136
00137             // Read grades
00138             vector<double> grades;
00139             double grade;
00140             while (iss » grade) {
00141               grades.push_back(grade);
00142             }
00143
00144             if (!grades.empty()) {
00145               student.setExamGrades(grades.back());
00146               grades.pop_back();
00147               student.setGrades(grades);
00148               student.calculateFinalGradesAverageClass(student);
00149             }
00150
00151             return is;
00152           }
00153       ~StudentClass() {
00154             grades.clear();
00155             exam_grade = 0;
00156             final_grade = 0;
00157
00158           }
00159
00160
00161        private:
00162         string name;
00163         string surname;
00164         ::vector<double> grades;
00165         double exam_grade;
00166         mutable double final_grade;
00167     };
00168
00169
00170
00171 #endif //STUDENTCLASS_H
```

6.16 StudentDeque.cpp Failo Nuoroda

```
#include "StudentDeque.h"
#include <ctime>
#include <fstream>
```

```
#include <sstream>
#include <iostream>
#include <numeric>
```

Funkcijos

- deque< StudentDeque > readFileDeque (int num)
- double averageDeque (const StudentDeque &student)
- void calculateFinalGradesAverageDeque (const StudentDeque &student)
- void sortDeque (deque< StudentDeque > &students, int num)
- void splitToGroupsDeque (deque< StudentDeque > &students, deque< StudentDeque > &kietekai, deque< StudentDeque > &vargsiukai, int num)
- void strategyTwoDeque (deque< StudentDeque > &students, deque< StudentDeque > &vargsiukai, int num)
- void strategyThreeDeque (deque< StudentDeque > &students, deque< StudentDeque > &vargsiukai, int num)

6.16.1 Funkcijos Dokumentacija

6.16.1.1 averageDeque()

```
double averageDeque (
            const StudentDeque & student)
```

6.16.1.2 calculateFinalGradesAverageDeque()

```
void calculateFinalGradesAverageDeque (
            const StudentDeque & student)
```

6.16.1.3 readFileDeque()

```
deque< StudentDeque > readFileDeque (
            int num)
```

6.16.1.4 sortDeque()

```
void sortDeque (
            deque< StudentDeque > & students,
            int num)
```

6.16.1.5 splitToGroupsDeque()

```
void splitToGroupsDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & kietekai,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.16.1.6 strategyThreeDeque()

```
void strategyThreeDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.16.1.7 strategyTwoDeque()

```
void strategyTwoDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.17 StudentDeque.h Failo Nuoroda

```
#include <deque>
#include <string>
```

Klasės

- struct StudentDeque

Funkcijos

- deque< StudentDeque > readFileDeque (int num)
- double averageDeque (const StudentDeque &student)
- void calculateFinalGradesAverageDeque (const StudentDeque &student)
- void sortDeque (deque< StudentDeque > &students, int num)
- void splitToGroupsDeque (deque< StudentDeque > &students, deque< StudentDeque > &kietekai, deque< StudentDeque > &vargsiukai, int num)
- void strategyTwoDeque (deque< StudentDeque > &students, deque< StudentDeque > &vargsiukai, int num)
- void strategyThreeDeque (deque< StudentDeque > &students, deque< StudentDeque > &vargsiukai, int num)

6.17.1 Funkcijos Dokumentacija

6.17.1.1 averageDeque()

```
double averageDeque (
            const StudentDeque & student)
```

6.17.1.2 calculateFinalGradesAverageDeque()

```
void calculateFinalGradesAverageDeque (
            const StudentDeque & student)
```

6.17.1.3 readFileDeque()

```
deque< StudentDeque > readFileDeque (
            int num)
```

6.17.1.4 sortDeque()

```
void sortDeque (
            deque< StudentDeque > & students,
            int num)
```

6.17.1.5 splitToGroupsDeque()

```
void splitToGroupsDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & kietekai,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.17.1.6 strategyThreeDeque()

```
void strategyThreeDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.17.1.7 strategyTwoDeque()

```
void strategyTwoDeque (
            deque< StudentDeque > & students,
            deque< StudentDeque > & vargsiukai,
            int num)
```

6.18 StudentDeque.h

Eiti į šio failo dokumentaciją.

```
00001 //
00002 // Created by Anastasija Fedorenko on 2025-03-12.
00003 //
00004
00005 #ifndef STUDENTDEQUE_H
00006 #define STUDENTDEQUE_H
00007
00008 #include <deque>
00009 #include <string>
00010 using namespace std;
00011
00012 struct StudentDeque {
00013     std::string name;
00014     std::string surname;
00015     std::deque<double> grades;
00016     int exam_grade;
00017     mutable double final_grade;
00018
```

```
00019 };
00020
00021 deque<StudentDeque> readFileDeque(int num);
00022
00023 double averageDeque(const StudentDeque &student);
00024 void calculateFinalGradesAverageDeque(const StudentDeque &student);
00025 void sortDeque(deque<StudentDeque>& students, int num);
00026 void splitToGroupsDeque(deque<StudentDeque>& students, deque<StudentDeque>& kietekai,
      deque<StudentDeque>& vargsiukai, int num);
00027 void strategyTwoDeque(deque<StudentDeque>& students, deque<StudentDeque>& vargsiukai, int num);
00028 void strategyThreeDeque(deque<StudentDeque>& students, deque<StudentDeque>& vargsiukai, int num);
00029
00030
00031 #endif //STUDENTDEQUE_H
```

6.19 StudentList.cpp Failo Nuoroda

```
#include "StudentList.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <numeric>
#include <ctime>
```

Funkcijos

- list< StudentList > readFileLists (int num)
- double averageList (const StudentList &student)
- void calculateFinalGradesAverageList (const StudentList &student)
- list< StudentList > sortList (list< StudentList > students, int num)
- void splitInTwo (list< StudentList > &students, int num)
- void strategyTwoList (list< StudentList > &students, list< StudentList > &vargsiukai, int num)
- void strategyThreeList (list< StudentList > &students, list< StudentList > &vargsiukai, int num)

6.19.1 Funkcijos Dokumentacija

6.19.1.1 averageList()

```
double averageList (
            const StudentList & student)
```

6.19.1.2 calculateFinalGradesAverageList()

```
void calculateFinalGradesAverageList (
            const StudentList & student)
```

6.19.1.3 readFileLists()

```
list< StudentList > readFileLists (
            int num)
```

6.19.1.4 sortList()

```
list< StudentList > sortList (
            list< StudentList > students,
            int num)
```

6.19.1.5 splitInTwo()

```
void splitInTwo (
            list< StudentList > & students,
            int num)
```

6.19.1.6 strategyThreeList()

```
void strategyThreeList (
            list< StudentList > & students,
            list< StudentList > & vargsiukai,
            int num)
```

6.19.1.7 strategyTwoList()

```
void strategyTwoList (
            list< StudentList > & students,
            list< StudentList > & vargsiukai,
            int num)
```

6.20 StudentList.h Failo Nuoroda

```
#include <string>
#include <list>
```

Klasės

- struct StudentList

Funkcijos

- list< StudentList > readFileLists (int num)
- double averageList (const StudentList &student)
- void calculateFinalGradesAverageList (const StudentList &student)
- list< StudentList > sortList (list< StudentList > students, int num)
- void splitInTwo (list< StudentList > &students, int num)
- void strategyTwoList (list< StudentList > &students, list< StudentList > &vargsiukai, int num)
- void strategyThreeList (list< StudentList > &students, list< StudentList > &vargsiukai, int num)

6.20.1 Funkcijos Dokumentacija

6.20.1.1 averageList()

```
double averageList (
            const StudentList & student)
```

6.20.1.2 calculateFinalGradesAverageList()

```
void calculateFinalGradesAverageList (
            const StudentList & student)
```

6.20.1.3 readFileLists()

```
list< StudentList > readFileLists (
            int num)
```

6.20.1.4 sortList()

```
list< StudentList > sortList (
            list< StudentList > students,
            int num)
```

6.20.1.5 splitInTwo()

```
void splitInTwo (
            list< StudentList > & students,
            int num)
```

6.20.1.6 strategyThreeList()

```
void strategyThreeList (
            list< StudentList > & students,
            list< StudentList > & vargsiukai,
            int num)
```

6.20.1.7 strategyTwoList()

```
void strategyTwoList (
            list< StudentList > & students,
            list< StudentList > & vargsiukai,
            int num)
```

6.21 StudentList.h

Eiti į šio failo dokumentaciją.

```
00001 //
00002 // Created by Anastasija Fedorenko on 2025-03-12.
00003 //
00004
00005 #ifndef STUDENTLIST_H
00006 #define STUDENTLIST_H
00007
00008 #include <string>
00009 #include <list>
00010 using namespace std;
00011
00012 struct StudentList {
00013     std::string name;
00014     std::string surname;
00015     std::list<double> grades;
00016     int exam_grade;
00017     mutable double final_grade;
00018
00019 };
00020 list<StudentList> readFileLists(int num);
00021 double averageList(const StudentList &student);
00022 void calculateFinalGradesAverageList(const StudentList &student);
00023 list<StudentList> sortList(list<StudentList> students, int num);
00024 void splitInTwo(list<StudentList> &students, int num);
00025 void strategyTwoList(list<StudentList> &students, list<StudentList> &vargsiukai,int num);
00026 void strategyThreeList(list<StudentList>& students, list<StudentList>& vargsiukai, int num);
00027
00028
00029 #endif //STUDENTLIST_H
```

Rodyklė